eSeMeshA: eBPF-based Service Mesh Acceleration for Cloud-Native Infrastructures

Arthur J Simas, Christian Esteve Rothenberg, Gergely Pongrácz
Universidade Estadual de Campinas, Brazil
Ericsson Research, Hungary
a249927@dac.unicamp.br, chesteve@dca.fee.unicamp.br, gergely.pongracz@ericsson.com

2025 IEEE 11th International Conference on Network Softwarization (NetSoft) | 979-8-3315-4345-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/NETSOFT64993.2025.11080556

***Abstract*—Service meshes are indispensable for enabling advanced communication features in cloud-native infrastructures. Though, due to the tight coupling of service meshes with the data path, they introduce significant overheads, resulting in degraded networking performance, including increased latency, poor throughput, and inefficient resource utilization. This paper presents an eBPF-based Service Mesh Acceleration framework, eSeMeshA, a proposal designed to mitigate these bottlenecks in intra-node communication by employing an in-kernel method to bypass costly data paths in modern service mesh architectures, such as Istio Ambient Mesh and Cilium, while also maintaining full support for legacy sidecar-based approaches, like Istio Sidecar. Experimental results show the potential of eSeMeshA to achieve significant gains, over 70% in throughput, up to 41.9% latency reduction, and 35.8% jitter improvement, all with minimal memory overhead of at most 30.4 MB and 23.2% less CPU usage, demonstrating its effectiveness as a scalable and efficient solution for cloud-native networking.**

***Index Terms*—Kubernetes, Service Mesh, eBPF, Acceleration, Cloud-Native**

I. Introduction

Service mesh has emerged as a key technology of cloud-native computing, enabling scalable, resilient, and flexible microservices architectures [1]. By abstracting service-to-service communication away from the application, service meshes enhance traffic management, security, and observability within distributed systems [2]. However, these benefits come at a significant performance cost: this layer of abstraction introduces non-negligible networking overheads that increase resource consumption and degrade network-related metrics, such as throughput and latency [3].

A key source of inefficiency lies in the middleware proxy model typically employed by service meshes, wherein a proxy container is deployed in-between services. While this design provides isolation and modularity, it introduces an extended data path: packets must traverse multiple user- and kernel-space transitions across containers and network interfaces. These repeated context switches and memory copies degrade performance. Recent architectural advancements aim to mitigate these issues [4]. Yet, such approaches remain constrained by the inherent limitations of the traditional kernel networking stack.

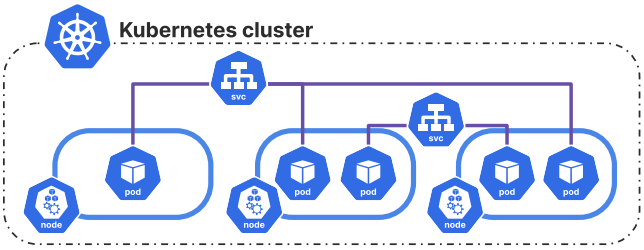

Figure 1: General architecture of a Kubernetes cluster.

This is where the extended Berkeley Packet Filter (eBPF) offers a paradigm shift. As cloud-native infrastructures adopt network softwarization, eBPF enables in-kernel packet processing at near-native speeds [5], bypassing the bottlenecks of conventional kernel subsystems and eliminating redundant transitions. eBPF's flexibility allows it to complement—rather than replace—existing service mesh architectures.

Building on these capabilities, this paper introduces `eSeMeshA`, an eBPF-based approach to transparently accelerate intra-node service mesh networking. By bypassing the kernel network stack and directly redirecting traffic between sockets, eSeMeshA reduces the overhead associated with service meshes, independent of the deployed architecture. The proposed solution is transparent to existing applications, requiring no modifications to code or cluster reconfigurations. Experimental results demonstrate its effectiveness: a 41.9% reduction in latency, a 35.8% lower jitter, and a 70.9% higher throughput, achieved with a memory overhead of just 30.4 MB and about 20% less CPU usage. These gains stem from three key contributions:

(i) **Literature Review.** Offers an overview of Kubernetes, service mesh, and eBPF technologies, alongside an analysis of related work, identifying gaps and opportunities for improvement in service mesh optimization;

(ii) **Architecture-Agnostic Design.** Presents a design that works transparently across different service mesh architectures, and without application or cluster modifications;

(iii) **Experimental Validation.** Provides a thorough evaluation of the proposed solution in a Kubernetes testbed.

The remainder is organized as follows: Section II provides a comprehensive background on Kubernetes, service mesh,

This work was supported by *Ericsson Telecomunicações LTDA*, and by the *São Paulo Research Foundation (FAPESP)* FAPESP, grant `2021/00199-8`, CPE SMARTNESS. This study was partially funded by *CAPES*, Brazil - Finance Code `001`.

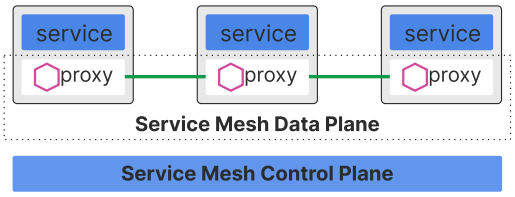

Figure 2: Sidecar-based service mesh architecture.

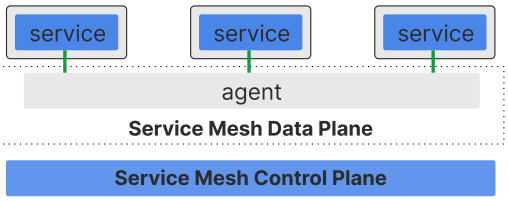

Figure 3: Sidecar-less service mesh architecture.

and eBPF; Section III presents eSeMeshA, detailing the general overview, design, and implementation; Section IV is focused on the experimental evaluation; Section V compares eSeMeshA against related work; Finally, Section VI and VII outlines future research and concludes the paper, respectively.

II. Background

A. Kubernetes

Manually managing containers at production-scale becomes impractical over time. This is where container orchestration with Kubernetes comes into play [1]. Kubernetes has become a core component in cloud-native environments, enabling the orchestration of containerized applications across distributed infrastructures. It automates tasks required to deploy, manage, and maintain containers in a distributed environment, ensuring consistency, availability, and efficiency [6]. The general architecture of a Kubernetes cluster, as represented in Fig. 1, is comprised of nodes, which host pods that encapsulate containers, and services that facilitate communication and load balancing between pods.

Kubernetes employs a flat networking model to enable seamless communication between pods, the smallest deployable units within the cluster [7]. Networking plugins based on the Container Network Interface (CNI) standard, such as Calico and Flannel, provide the necessary abstraction to implement these networking capabilities. While Kubernetes simplifies container management, its native networking model lacks advanced features such as fine-grained traffic control, security, and observability, leaving a gap in managing complex microservices architectures.

B. Service Mesh

Service mesh extends Kubernetes networking capabilities by providing advanced features like traffic management, service discovery, encryption, and observability [8]. It abstracts the complexities of inter-service communication by injecting middleware proxies alongside application containers. These proxies manage traffic and enforce policies without requiring changes to the application code.

The implementation of a service mesh using the sidecar-based architecture, depicted in Fig. 2, is popular because it is straightforward to build in Kubernetes, where each pod include the sidecar proxy as an additional container [2]. Despite its advantages, this approach introduces significant networking overhead, as packets traverse multiple hops between proxies and kernel stack layers, and consumes additional resources due to the deployment of a sidecar per pod.

In response to these challenges, sidecar-less architectures, such as Istio Ambient [4] using a shared agent per-node (called ztunnel), and Cilium [9] using eBPF to implement its features, have emerged to reduce resource consumption by using proxies at the node level or leveraging kernel-based mechanisms to eliminate sidecars, as illustrated in Fig. 3. However, both architectures remain constrained by the inefficiencies of the traditional kernel network stack, which limits their ability to fully optimize performance at scale.

C. eBPF

To address the inefficiencies of the kernel network stack, especially for performance-demanding applications, an in-kernel solution is needed. The extended Berkeley Packet Filter (eBPF) is a technology that enables safe, efficient execution of custom programs within the Linux kernel. By attaching to predefined hook points, eBPF programs can monitor, filter, or manipulate packets without the need to modify kernel source code, load kernel modules, or rely on user-space processing [5]. eBPF's versatility extends to various networking tasks, including load balancing, security enforcement, and packet redirection, making it a vital tool for optimizing modern infrastructures.

Unlike traditional kernel-based networking mechanisms, eBPF eliminates the need for frequent context switching between kernel- and user-space, significantly reducing latency and resource consumption. eBPF is particularly well-suited for optimizing microservices communication in Kubernetes and service mesh environments. By enabling in-kernel packet redirection and bypassing the kernel network stack, it provides means for achieving the goals of network softwarization—programmable, efficient, and scalable networking.

III. eSeMeshA: eBPF-based Service Mesh Acceleration

Our proposed solution called `eSeMeshA` leverages eBPF to address the performance bottlenecks inherent in service mesh architectures. By accelerating traffic using in-kernel-bypass, we achieve significant improvements in different network-related metrics across both sidecar-based and sidecar-less deployments. It is designed to be transparent and architecture-agnostic while delivering enhanced performance.

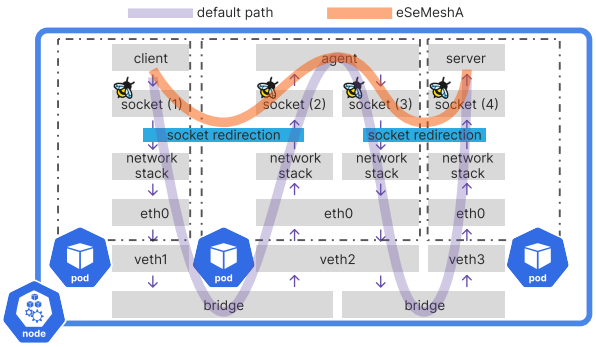

Figure 4: Packet path in a sidecar-less service mesh. The purple-ish line is the default path, and the orange-ish line is the accelerated path using eSeMeshA.

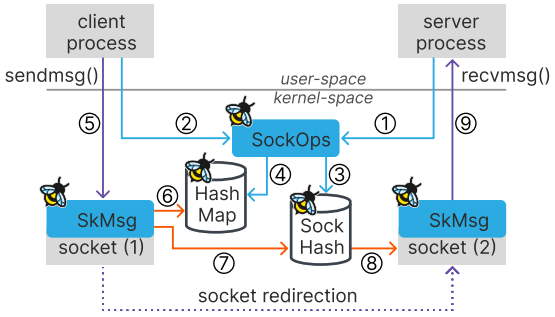

Figure 5: Socket redirection workflow using eBPF.

A. General Overview

The architecture of eSeMeshA focuses on optimizing the communication path between services by intercepting and redirecting packets directly within the kernel, thereby avoiding redundant traversals of the network stack and context switches between user/kernel-space. In default service mesh configurations, as shown in Fig. 4 (purple-ish line), packets follow an elongated path: they traverse multiple user/kernel-space hops, passing through network interfaces, virtual bridges, and middleware proxies, before reaching their destination. This inefficient routing increases latency and resource utilization, particularly in scenarios with high traffic.

The efficient packet redirection mechanism using eBPF of eSeMeshA is depicted in Fig. 4 (orange-ish line). Instead of relying on the traditional networking stack, eBPF programs intercept packets once they arrive at the kernel level and directly route them to their destination sockets. This design ensures that communication occurs without unnecessary intermediate hops, reducing latency and improving throughput. Additionally, eSeMeshA operates transparently, requiring no modifications to existing applications or service mesh configurations, making it compatible with both sidecar-based and sidecar-less deployments (*e.g.*, Istio Ambient, Cilium).

B. Design and Implementation

The high-level workflow of eSeMeshA, as depicted in Fig. 5, begins when the socket is created at user-space, both at the server-side ① and the client-side ②. They are detected by *SockOps*, which adds them to the eBPF Maps *SockHash* ③ and *HashMap* ④, responsible for the indexed socket storage. When a message is written to a socket stored in *SockHash*, it is captured by *SkMsg* ⑤, which retrieves the mapping for the other socket's end in *HashMap* ⑥, then *SockHash* is used to redirect the message ⑦ to the respective socket ⑧, delivering it to the server process ⑨.

Dual-stack IPv4 and IPv6 support: eSeMeshA internally uses only IPv6 addresses. To ensure support for both IP versions, IPv4 addresses are mapped to the IPv6 address space in the prefix `::FFFF:x.y.z.w`, where `x.y.z.w` represents the original IPv4 address, as per RFC 4291;

Socket Indexing Mechanism: Kubernetes Services abstraction uses a front-facing IP address to expose pods, which differs from the actual pod IP. Due to this, the socket's 4-tuple (source IP-port, destination IP-port) is different on the client-side from the server-side. In this situation, the indexing of sockets is possible by identifying the connection by the IP-port of the client-side (since the tuple on the server-side is common to all connections) paired with the side of the socket (*i.e.* client or server), forming a key **`sock_id{IP,port,side}`**;

Socket Capture and Storage: eSeMeshA uses a *SockOps* program to monitor TCP socket events such as connection establishment. When a socket is created within the relevant control group (cgroup), the *SockOps* program captures it and stores the socket reference in eBPF maps to enable the efficient indexing and retrieval of active sockets based on user-defined keys. The eBPF *SockHash* Map stores sockets identified by the **`sock_id`** key, with the `side` field determined by the socket event operation: `BPF_SOCK_OPS_ACTIVE_ESTABLISHED_CB` for client-side and `BPF_SOCK_OPS_PASSIVE_ESTABLISHED_CB` for server-side. Then, *HashMap* stores the mapping of the socket TCP 4-tuple to the corresponding **`sock_id`** of the other side of the connection;

Packet Interception and Redirection: Once sockets are captured, the *SkMsg* program attached to *SockHash* is triggered by new message events. This program intercepts packets sent through the associated stored socket and uses the socket indexing mechanism to redirect them directly to the destination socket within the same node, bypassing the kernel's standard networking stack;

Rust framework: The implementation is based in the Rust Language, leveraging its performance and memory safety, both critical for eBPF programming. We also use the Aya framework for streamlined eBPF development, and the crates (*i.e.* Rust 'libraries') `bpf-linker` and `bindgen-cli` for compatibility across various Linux kernel versions.

IV. Experimental Evaluation

A. Setup

The experiments were performed in a virtualized environment provisioned by LXD, a lightweight container hypervisor. The VM was configured with 4 vCPUs @ 2.2 GHz, 8 GB of memory, image ubuntu:24.04, and Linux kernel v6.8.0-49-generic. The single-node Kubernetes cluster was deployed using the RKE2 distribution v1.31.4-rke2r1, and CNI Calico v3.29.1 (except for the Cilium service mesh).

The *ping-echo* workload consists of a high-frequency ping client and an echo server, both implemented in Python. The client sends a packet of 24 bytes to the server over TCP. It was executed both with the eSeMeshA optimization disabled (baseline) and enabled across different service mesh setups, including no service mesh (*none*), Cilium v1.16.4 (`cilium`), Istio Ambient v1.24.2 (`istio-ambient`), and Istio Sidecar v1.24.2 (`istio-sidecar`).

B. Results

The results of our evaluation are summarized in Table I. They show a latency reduction on the order of 41.9% for the best scenario (*istio-sidecar*), whilst the *worst* reduction was of 22.9% (*cilium*). Our proposal successfully reduced latency across all scenarios, as shown in Fig. 6.

We observe a jitter reduction, leading to better latency stability. The *istio-sidecar* scenario experienced the largest improvement of 35.8%, and *cilium* observed a slight increase in jitter of 15.2%.

Observing the throughput performance in Fig. 7 measured by requests per second (RPS), followed a constant upward trend with the use of eSeMeshA. Optimized scenarios consistently outperformed their unoptimized counterparts, with gains of at most 70.9% in the best case (*istio-sidecar*).

CPU usage, measured by the sum of system and user time, is also positively impacted with the usage of eSeMeshA, as depicted in Fig. 8. When enabled, CPU usage decreases by at least 12.1% for *cilium*, and at most 23.2% for *istio-sidecar*.

With regard to memory usage, eSeMeshA exhibits a small memory overhead when enabled, ranging from an increase of 17 MB (*istio-ambient*) to 30.4 MB (no service mesh).

V. Related Work

The complexity and performance overheads associated with service mesh implementations have driven other researchers into optimization strategies.

SPRIGHT [10] is a serverless framework that utilizes shared memory and eBPF to optimize data plane operations. By bypassing the kernel stack, it achieves performance gains including a 5× improvement in throughput, a 53× reduction in latency, and a 27× reduction in CPU usage compared to existing solutions like Knative. Despite its efficiency, SPRIGHT does not provide native support for service mesh features such as traffic management, security, or observability, as it is focused in the serverless use-case.

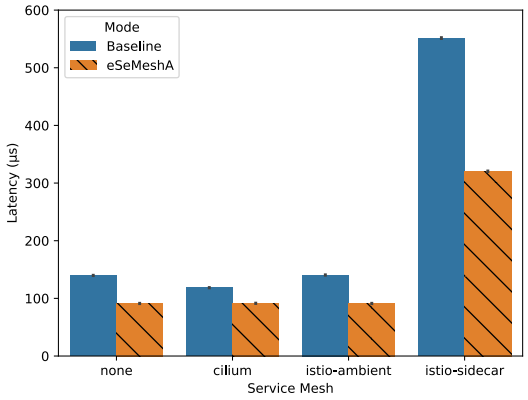

Figure 6: Latency of *ping-echo* across different scenarios.

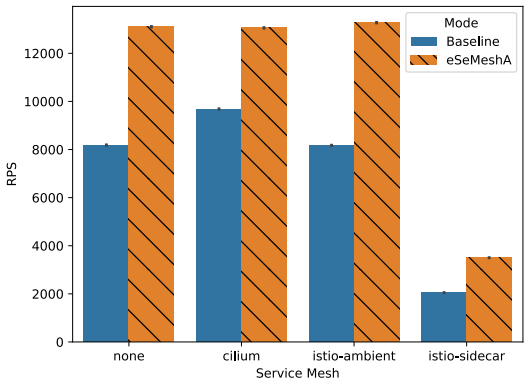

Figure 7: RPS of *ping-echo* across different scenarios.

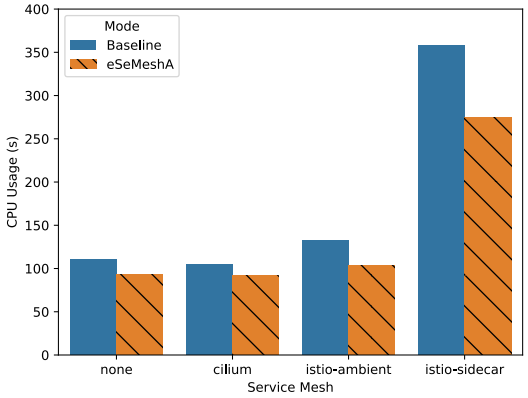

Figure 8: CPU usage of *ping-echo* across different scenarios.

Cilium [9] is a sidecar-less service mesh that heavily uses eBPF to implement its CNI plugin, providing networking, security, and observability without the need for dedicated proxy containers. It significantly reduces the overhead associated with service-to-service communication, although it requires the usage of its CNI plugin, limiting its adoption in clusters already configured with other CNIs.

Table I: Summary of results. Average values are shown.

Service Mesh	Mode	Latency (µs)	Jitter (µs)	RPS	CPU (s)	Memory (MB)
none	Baseline	139.8	96.1	8186.1	110.5	1380.2
	eSeMeshA	91.2 (**-34.8%**)	94.8 (**-1.4%**)	13104.6 (**60.1%**)	92.6 (**-16.2%**)	1410.6 (**2.2%**)
`cilium`	Baseline	118.5	88.0	9685.2	104.9	1325.7
	eSeMeshA	91.4 (**-22.9%**)	101.7 (**15.6%**)	13058.0 (**34.8%**)	92.3 (**-12.1%**)	1351.8 (**2.0%**)
`istio-ambient`	Baseline	140.5	101.8	8172.3	132.2	1660.7
	eSeMeshA	91.1 (**-35.2%**)	113.4 (**11.4%**)	13274.4 (**62.4%**)	103.2 (**-21.9%**)	1677.7 (**1.0%**)
`istio-sidecar`	Baseline	551.5	278.9	2049.1	357.8	1645.7
	eSeMeshA	320.4 (**-41.9%**)	179.0 (**-35.8%**)	3502.1 (**70.9%**)	274.7 (**-23.2%**)	1667.6 (**1.3%**)

The work Yang *et al.* [3] introduces an eBPF-based optimization targeting Istio's sidecar-based mode. Their approach reduces request latency by up to 21% for 90% of requests, alongside slight reductions in CPU and memory usage. While effective for Istio in its default sidecar-based configuration, the solution is limited to this specific architecture, lacks support for sidecar-less deployments, and is IPv4-only, restricting its generalizability.

An earlier iteration of the proposed approach, eZtunnel [11] (not to be confused with *ztunnel*, an Istio Ambient component), transparently offloads networking traffic, achieving improvements in figures of over 20% in median latency reduction and almost 10% in jitter decrease. However, while effective, eZtunnel is limited in its scope and optimization capabilities, neither addressing support for IPv6, nor assessing the support for other service mesh architectures.

Although these approaches illustrate significant advancements in addressing the performance challenges, limitations still remain, such as reliance on specific architectures, lack of integration with service mesh features, and incompatibility with existing infrastructures.

Our `eSeMeshA` proposal builds on these insights by leveraging eBPF to provide architecture-agnostic, service-mesh-aware optimizations that seamlessly integrate into existing Kubernetes environments. The experimental results demonstrate the advantages in optimizing service mesh networking across diverse scenarios. It consistently reduces latency and improves throughput. Notably, performance gains were achieved without significant resource overhead, consuming a small memory footprint. The CPU usage reduction correlates with eSeMeshA's in-kernel bypass mechanism, which minimizes context switching and kernel stack traversal—key contributors to CPU overhead in traditional service mesh data paths.

VI. Future Perspectives

Experimental results present eSeMeshA as a viable solution, but it requires further assessment, such as understanding the integration complexity with production environments while investigating the possible security implications and service mesh effectiveness when bypassing the default kernel network stack. Additional research will also explore the solution to support inter-node communication and evaluate its impact in multi-node architectures. Furthermore, we plan to evaluate more metrics with diversified workloads, such as a 5G core.

VII. Conclusions

We presented `eSeMeshA`, an eBPF-based solution to accelerate service mesh networking. The experimental evaluation was conducted across various service mesh configurations, including sidecar-based and sidecar-less deployments. It demonstrated significant improvements in latency, better latency stability, and improved throughput, while consuming less CPU and a small memory footprint.

Artifacts

Code repository and benchmark results of this research are available at: `github.com/smartness2030/eSeMeshA`

References

[1] D. Gannon, R. Barga, and N. Sundaresan, "Cloud-native applications," *IEEE Cloud Computing*, vol. 4, no. 5, pp. 16–21, 2017.

[2] L. Calcote and Z. Butcher, *Istio: Up and Running. Using a Service Mesh to Connect, Secure, Control, and Observe*. O'Reilly Media, 2019.

[3] W. Yang, P. Chen, G. Yu, *et al.*, "Network shortcut in data plane of service mesh with ebpf," *Journal of Network and Computer Applications*, vol. 222, Feb. 2024.

[4] J. Howard, E. J. Jackson, Y. Kohavi, *et al.* "Introducing ambient mesh: A new dataplane mode for istio without sidecars," Istio Authors. (2022), [Online]. Available: https : / / istio . io / latest / blog / 2022 / introducing-ambient-mesh/.

[5] M. A. M. Vieira, M. S. Castanho, R. D. G. Pacífico, *et al.*, "Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications," *ACM Comput. Surv.*, 2020.

[6] "What is kubernetes?" Google. (), [Online]. Available: https://cloud.google.com/learn/what-is-kubernetes.

[7] "Cluster networking," Kubernetes. (), [Online]. Available: https : // kubernetes.io/docs/concepts/cluster-administration/networking/.

[8] "Service mesh – cloud native glossary," Cloud Native Compute Foundation. (), [Online]. Available: https://glossary.cncf.io/service-mesh/.

[9] "What is Cilium?" Cilium Authors. (), [Online]. Available: https : //cilium.io/get-started/.

[10] S. Qi, L. Monis, Z. Zeng, *et al.*, "Spright: Extracting the server from serverless computing! high-performance ebpf-based event-driven, shared-memory processing," in *Proceedings of the ACM SIGCOMM 2022 Conference*.

[11] A. J. Simas, F. E. Rodriguez Cesen, and C. E. Rothenberg, "eZtunnel: Leveraging eBPF to Transparently Offload Service Mesh Data Plane Networking," in *2024 IEEE 13th International Conference on Cloud Networking (CloudNet)*.